

SKILLSPRINT AI

Professional Project Documentation

Source-Grounded Employee Onboarding Intelligence

Based on SRS v1.0 and the supplied implementation repository



Generative AI • Python Validation • Traceability • RBAC • Learning Analytics

Prepared: 28 September 2026

Software Requirements Specification Baseline: Version 1.0

Prepared from the supplied SRS and implementation repository

Document Control

Item	Value
Project	SkillSprint AI
Theme	OnboardVerse
Category	Generative AI PowerPlay
Document	Professional Project Documentation
Version	Version 1.0
Prepared	28 September 2026
Baseline	SkillSprint AI SRS Version 1.0
Audience	Project team, evaluator, developers, reviewers, HR/training stakeholders

Table of Contents

Document Control	2
Table of Contents	2
1. Executive Summary	4
2. Project Background and Problem Statement	4
3. Objectives	4
4. Scope	4
5. System Overview	5
6. Technology Stack	5
7. Architecture	5
8. User Roles and Access Control	5
9. Document Management and Processing	6
10. Role Requirement Matrix	6
11. GenAI Generation Pipeline	6
12. Independent Python Validation Pipeline	7
13. Verification Metrics	7
14. Learning Experience	7
15. Human Review and Audit	7
16. Policy Update and Impact Analysis	7
17. Security and Prompt-Injection Protection	7
18. Database Design	8
19. API and Backend Design	8
20. Frontend and User Interface	8
21. Testing and Quality Assurance	8
22. SRS Compliance and Traceability	9
23. SRS Compliance Matrix — Key Areas	9
24. Installation and Execution	9
25. Project Deliverables	9

26. Current Gaps / Evaluation Readiness 10

27. Future Enhancements 10

28. Conclusion 10

Appendix A — Architecture Diagrams 11

Appendix B — Supplied Dataset and Roles 13

 Supported roles 13

Appendix C — Representative API Surface 14

Appendix D — Core Data Entities 15

Appendix E — Final Submission Readiness Checklist 16

1. Executive Summary

SkillSprint AI is a web-based employee onboarding intelligence platform designed around the requirements of the supplied SkillSprint AI Software Requirements Specification (SRS), Version 1.0. The system converts approved organizational documents into role-specific onboarding plans and then subjects those plans to an independent deterministic Python validation pipeline.

The central architectural principle is separation of responsibilities. Generative AI is used for personalization, structured plan generation and learning-content enrichment. It is not treated as the authority for company policy. The approved source documents and Role Requirement Matrix remain the ground truth, while Python rules independently check coverage, traceability, role alignment, source validity, duplicate content, learning sequence, quiz mappings, rubric weights and contradiction conditions.

The supplied implementation uses Flask as the web backend, HTML/CSS/JavaScript for the web interface, SQLite as the primary local data store, optional MongoDB Atlas synchronization, Python document-processing libraries, and OpenAI-compatible provider integration for GenAI. The repository also contains prompts, tests, security evidence, installation/execution guides, a sample organizational dataset, and compliance/export functions.

Repository inspection confirms a substantial implementation aligned with the SRS. The supplied dataset contains 23 DOCX documents, the local database snapshot contains 150 requirements with 100 marked mandatory, and the code defines 10 supported job roles and six onboarding stages. The inspected local database contains two saved plans and a small current user/employee snapshot; this is runtime-state evidence rather than a limitation of the role catalog or application design.

2. Project Background and Problem Statement

Corporate onboarding information is typically distributed across handbooks, HR policies, SOPs, role descriptions, process manuals, FAQs and compliance instructions. Manual conversion of these documents into individualized training plans is time-consuming and can become inconsistent when documents change.

The project addresses this problem by treating organizational documentation as the source of truth and by attaching traceability metadata to generated learning content. Instead of returning a free-form AI answer, the application creates structured records that can be compared against independently stored requirements.

The SRS specifically requires a dual-pipeline approach: a Python GenAI Generation Pipeline and a separate Python Ground-Truth Validation Pipeline. The implementation follows this model by separating structured generation from deterministic validation and by persisting generation metadata, validation reports and review decisions.

3. Objectives

Automate the preparation of personalized employee onboarding plans from approved organizational documents.

Support multiple job roles and role-specific requirements rather than assigning a single generic training plan to every employee.

Preserve source traceability from generated modules, tasks and quizzes back to document and section references.

Independently validate AI-generated output using deterministic Python rules and a Role Requirement Matrix.

Detect missing, unsupported, outdated, duplicated, contradictory or incorrectly sequenced learning content.

Provide administrators, managers and reviewers with approval, audit and impact-analysis workflows.

Give employees a structured learning portal with progress tracking, quizzes, tasks, evidence submission and recommendations.

Provide compliance-oriented exports and evidence suitable for project evaluation.

4. Scope

The implemented scope covers user authentication and role-based access control; employee and role management; organizational document upload, validation, parsing and chunking; requirement extraction and matrix storage; structured

onboarding generation; deterministic validation; human review; employee learning; progress and recommendations; policy-impact tracking; selective regeneration; search/filtering and reporting.

The SRS explicitly places live HRMS, payroll, enterprise identity-management systems and commercial LMS integration outside the mandatory scope. The supplied implementation therefore focuses on a self-contained onboarding platform rather than attempting to become an HR/payroll replacement.

Supported upload formats in the implementation include PDF, DOCX, TXT, Markdown and CSV. PDF and DOCX are the mandatory formats identified for the hidden evaluation pack in the SRS.

5. System Overview

The application follows a source-grounded lifecycle: document intake → validation → parsing → chunking → approval → requirement extraction → role-specific generation → independent validation → human review → employee release → learning/progress tracking → policy impact analysis and regeneration.

The system distinguishes between source evidence and generated learning language. The validator compares structured identifiers and attributes such as requirement ID, role, mandatory status, source document, source section, priority and due stage. This is consistent with the SRS rule that the two pipelines are not expected to generate identical natural-language sentences.

6. Technology Stack

Backend: Python and Flask.

Frontend: HTML5, CSS3 and JavaScript, with dedicated landing, login, admin CRM and employee portal interfaces.

Primary database: SQLite for the local/self-contained application. Optional MongoDB Atlas synchronization is implemented through a separate storage module.

Document processing: pypdf for PDF extraction and python-docx for DOCX processing; text/Markdown/CSV support is also included in the upload allow-list.

AI integration: OpenAI-compatible client configuration supporting Groq, OpenRouter and OpenAI provider modes. The repository documentation identifies Groq as the native demonstration provider.

Validation: custom Python rule engine with structured JSON/schema checks and deterministic business rules.

Reporting: CSV, Excel-compatible output through openpyxl, and PDF reports through ReportLab.

Security: Flask sessions, password hashing, RBAC decorators, upload validation, prompt-injection scanning, review gates and environment-based secret configuration.

Version control/deployment: Git/GitHub is part of the SRS delivery requirements; the repository contains README, tests, documentation and configuration guidance.

7. Architecture

The architecture is intentionally modular even though the competition implementation is centered in a Flask application. `app.py` contains authentication, database helpers, document processing, generation, validation and API routes. Static files provide the login, admin and employee experiences. Prompt templates are stored separately under `prompts/`, while tests and documentation are kept in dedicated directories.

The most important architectural boundary is between Pipeline 1 and Pipeline 2. Pipeline 1 can call a configured GenAI provider or use a structured fallback when the provider is unavailable. Pipeline 2 does not approve the result because an AI model says it is correct; it independently evaluates the result against active requirements and source metadata.

Optional MongoDB synchronization mirrors the local application state when configured. The code is designed to keep local operation available if the remote synchronization layer is unavailable.

8. User Roles and Access Control

Administrator: manages users, documents, roles, onboarding plans, compliance information, reviews and system-level operations.

Manager: generates and monitors onboarding plans, reviews progress and participates in policy-impact workflows.

Reviewer: reviews source documents and generated plans and can approve, reject, request changes or apply authorized overrides.

Employee: accesses only the employee portal and their own learning information, submits evidence, completes learning activities and requests leave.

The backend enforces authorization with API and page decorators. Frontend visibility is not treated as the security boundary; protected API routes verify the authenticated role on the server.

9. Document Management and Processing

Authorized users upload organizational documents with metadata such as title, category, version, effective date and department. The application applies an upload size limit, checks allowed extensions, detects duplicate title/version pairs, extracts text and stores source content.

Documents enter a review state before they become approved evidence. Prompt-injection indicators can place a document into security review and prevent normal approval. This protects the requirement matrix from being built automatically from untrusted source material.

Document chunks preserve identifiers, headings/section-like references and source text. The application also retains version information and document metadata, allowing validation to reject or flag obsolete sources.

The supplied dataset contains 23 DOCX files, including company handbook, HR, security, privacy, SOP, FAQ, role-description and adversarial material. It also contains an information-security version pair in which Version 2 is active and Version 1 is obsolete, providing a concrete version-control scenario.

10. Role Requirement Matrix

The Role Requirement Matrix is the independent business reference used by the validator. Each requirement stores an identifier, role, description, mandatory/optional classification, priority, due stage, source document, source section and assessment information. Additional metadata records learning classification and competency.

The repository's local database snapshot contains 150 requirements, of which 100 are marked mandatory. The source-code role catalog contains ten supported roles: Sales Executive, Customer Support Executive, HR Executive, Finance Associate, Operations Coordinator, Marketing Executive, Software Support Engineer, Branch Manager, Data Analyst and Team Leader.

The extraction logic recognizes controlled requirement language such as must, mandatory, required, shall, recommended and optional. It also derives classifications such as Must Complete, Must Demonstrate, Must Know and Recommended. This creates a structured bridge from source documents to learning requirements.

11. GenAI Generation Pipeline

The first pipeline generates a structured onboarding plan from employee context, role requirements and approved source evidence. The expected output is JSON rather than uncontrolled prose.

The output schema includes employee information and generated items with requirement ID, role, module, mandatory status, source document ID, source section ID, priority, due stage, objective, task, difficulty, prerequisites, checklist, scenario, assessment, quiz and rubric information.

Prompt templates are versioned and stored outside the main application logic. Generation metadata records provider, model, prompt version, attempts and fallback behavior. Invalid or incomplete AI output is checked before it can become the final plan.

The implementation also includes a deterministic structured-template fallback. This allows the application to demonstrate the end-to-end pipeline without depending completely on an external API connection. The fallback is labelled in generation metadata and is still subjected to the same validation stage.

12. Independent Python Validation Pipeline

The second pipeline is deterministic Python logic. It checks whether each generated item maps to a real requirement for the selected role, whether the mandatory status and source identifiers match, whether the source is active, and whether duplicate requirement IDs appear.

Additional checks cover learning sequence and prerequisites, quiz/source alignment, assessment rubric weights, contradiction conditions, unsupported requirements and overall coverage/traceability metrics.

The final validation status can be Verified, Verified with Warning, Incomplete, Unsupported, Contradictory or Manual Review Required. Human review remains available when the system cannot safely resolve an issue.

This approach directly addresses the SRS rule that GenAI must not replace Python validation, business rules, source traceability, schema validation, comparison logic or audit logic.

13. Verification Metrics

Mandatory Requirement Coverage Score = Covered Mandatory Requirements / Total Mandatory Requirements × 100.

Traceability Score = percentage of generated items with valid source references, as calculated by the implementation's validation routine.

The validation report also records missing requirement count, unsupported requirement count, issue details and verification status. These values provide evidence for debugging and evaluator review.

In the inspected database snapshot, both saved plans have a 100% coverage score and 100% traceability score in their persisted reports. Their stored plan status is Verified; the underlying review/report state should still be interpreted together with the recorded review workflow rather than as a substitute for human review evidence.

14. Learning Experience

The employee portal presents a six-stage roadmap: Day 1, Week 1, Week 2, First 30 Days, First 60 Days and First 90 Days. Modules can expose objectives, tasks, source citations, quizzes and completion controls.

Progress is recorded against employee/requirement pairs. Quiz or learning attempts can store scores and status, and employee learning status can be derived as Completed, Requires Attention, On Track or Assessment Required.

Adaptive recommendations can be generated from weak areas, quiz/assessment performance and incomplete work. The employee portal also supports evidence submission and leave-request workflows.

15. Human Review and Audit

Administrators, managers and reviewers can inspect generated plans and make decisions including Approved, Rejected, Needs Changes and Override. Review records store the decision, comments, reviewer and optional override data.

The review workflow is important because a deterministic validator can identify a mismatch without always having enough organizational context to resolve it. The application therefore keeps the original result and the human decision as separate audit evidence.

16. Policy Update and Impact Analysis

When source documents change, the system can create policy-impact records linking a changed document to affected plans and requirements. Administrators can inspect impact records and regenerate affected plans rather than treating every plan as permanently valid.

This design supports the SRS requirement for selective regeneration: content affected by a policy update can be regenerated while unrelated content remains unchanged. The workflow creates a new draft for review rather than silently replacing an approved employee plan.

17. Security and Prompt-Injection Protection

The system treats uploaded documents as untrusted data. A regex-based injection detector identifies common phrases such as instructions to ignore previous rules, reveal prompts or act as ChatGPT. Flagged documents are prevented from normal approval.

Security testing documented in the repository covers unauthorized API access, cross-employee access, invalid extensions, empty/corrupt files, duplicate documents, invalid metadata dates, prompt injection, obsolete sources, plan release gates, API-key handling, unauthenticated access and account deactivation controls.

Secrets are intended to remain in `.env`, which is excluded from Git. The documentation explicitly warns against committing API keys. A production deployment should additionally use a managed secret store, encrypted object storage, malware scanning, rate limiting and centralized audit logging.

18. Database Design

The SQLite schema contains separate entities for documents, chunks, requirements, employees, plans, completions, users, document submissions, document-role mapping, events, announcements, leave requests, generation logs, plan reviews, policy impacts, learning attempts, requirement metadata, document metadata and employee profiles.

The schema reflects the project's auditability goal: plans retain generated JSON and validation reports; generation logs retain provider/model/prompt information; review records retain human decisions; policy-impact records connect document changes to affected plans; and completions/learning attempts preserve employee progress.

19. API and Backend Design

The backend exposes authentication, user management, dashboard/analytics, document, requirement, plan, review, learning, recommendation, impact-analysis, system-status, GenAI test and export endpoints.

Representative endpoints include `/api/auth/login`, `/api/documents/upload`, `/api/documents/approve`, `/api/generate`, `/api/plans/review`, `/api/employee/dashboard`, `/api/learning-attempts`, `/api/impact-analysis`, `/api/impact-analysis/regenerate`, `/api/export/compliance.csv`, `/api/export/compliance.xlsx` and `/api/export/compliance.pdf`.

Protected endpoints use server-side authorization checks. Employee-specific endpoints additionally verify that the authenticated employee is requesting only their own records.

20. Frontend and User Interface

The repository contains dedicated pages for landing, login, administrator CRM and employee learning. JavaScript modules load API data, render employee/document/plan registers and handle interactive learning actions.

The admin workspace is designed around operational tabs such as employees, documents, requirements, plans, impact analysis and leave requests. The employee portal focuses on the personal roadmap, progress, quizzes, recommendations and evidence submission.

The UI is responsive and separates presentation from backend authorization. The project uses CSS files for the login/employee experience and CRM-specific styling for administrative operations.

21. Testing and Quality Assurance

The repository contains automated tests in `tests/test_srs_controls.py` covering requirement classification, unsupported-item validation, supported formats, role retrieval, public registration and admin employee creation.

The repository also contains a dedicated security test report with 16 documented security cases, including access-control and upload/security scenarios.

During this documentation build, the supplied test suite was not executed to completion in the isolated documentation environment because Flask was not installed in that environment. This is an environment limitation, not a claim that the project's test cases are invalid. The project itself contains installation instructions for creating the correct virtual environment and installing `requirements.txt` before running the tests.

For final evaluation, the team should execute the complete test suite inside the project's configured virtual environment and retain the console result or CI evidence.

22. SRS Compliance and Traceability

The implementation aligns with the major SRS themes: document ingestion, role matrix, structured generation, independent validation, traceability, coverage scoring, security, review workflow, employee dashboard, progress tracking, impact analysis, exports and installation/execution documentation.

The repository includes `SRS\_COMPLETION\_MATRIX.md`, which maps major SRS controls to implementation evidence. It also includes `FINAL\_SUBMISSION\_CHECKLIST.md`, which separates completed repository artifacts from remaining competition/evaluation evidence.

23. SRS Compliance Matrix — Key Areas

SRS area	Project implementation evidence	Status
Document intake & parsing	PDF/DOCX mandatory support; TXT/MD/CSV allow-list; metadata, size and duplicate checks; text extraction and chunks.	Implemented
Role Requirement Matrix	150 requirement records in inspected DB snapshot; 100 mandatory; ten role catalog in source.	Implemented
Structured GenAI output	JSON schema, versioned prompt, generation logs, provider/model metadata and fallback.	Implemented
Independent validation	Deterministic Python checks for IDs, role, mandatory state, sources, duplicates, sequence, quizzes and rubrics.	Implemented
Coverage & traceability	Coverage and traceability scores persisted in plan validation reports.	Implemented
Security / injection	Injection scanning, security-review state, RBAC and protected API/page decorators.	Implemented
Human review	Approve/Reject/Needs Changes/Override plus review audit records.	Implemented
Employee learning	Six-stage roadmap, progress, learning attempts, recommendations and evidence submission.	Implemented
Policy impact	Impact records and selective regeneration endpoint.	Implemented
Reports	CSV, Excel-compatible and PDF compliance exports.	Implemented
Testing	Automated SRS-control tests plus 16 documented security cases.	Implemented; rerun in project venv
10-role evidence	Role catalog and role-description dataset support ten roles; current DB snapshot contains fewer active employees/plans.	Evidence to refresh
100+ comparison evidence	Validation/export mechanisms exist; final retained comparison dataset should be produced.	Evidence to refresh
Deployment + demo video	Documentation/checklist exists; repository marks these as pending.	Pending
Team contribution record	SRS requires it; repository checklist references team responsibility.	Pending/Team action

24. Installation and Execution

The intended installation flow is: install Python, create a virtual environment, activate it, install `requirements.txt`, configure `.env`, optionally configure MongoDB, then run `python app.py` and open the local Flask URL.

The repository's installation guide documents the same workflow and includes troubleshooting for missing dependencies, port conflicts, AI configuration, database locking, MongoDB connectivity and DOCX parsing.

API keys must not be committed to a public GitHub repository. The `.env` file is explicitly excluded through `.gitignore`.

25. Project Deliverables

The repository contains source code, README, requirements, license, installation and execution guides, project report material, AI usage documentation, technical blog material, SRS completion matrix, security testing report, test cases,

prompt templates, dataset documents and export functionality.

The SRS additionally requires a deployed application URL, demonstration video, technical blog, AI usage declaration and team contribution record. The repository checklist identifies the demo video and deployment URL as remaining team responsibilities at the time of inspection.

26. Current Gaps / Evaluation Readiness

The application implementation is broad, but final evaluation evidence should be treated separately from feature existence. The inspected database snapshot currently contains two saved plans and one employee record, even though the application is designed around ten roles and the dataset contains ten role-description documents. Before submission, the team should generate and preserve a complete plan set covering all ten roles.

The SRS asks for 100+ requirement-level GenAI/Python comparison results. The repository states that this evidence is exportable, but the final submission should retain a concrete comparison export showing the required volume.

The repository checklist marks deployment and the mandatory demonstration video as pending. These should be completed before the final submission.

The project contains both `NovaTech Solutions` references in the dataset and a `Northstar Industries` reference in the technical blog. For a polished final submission, standardize the fictional company naming across documentation, screenshots and dataset references.

27. Future Enhancements

Move from a local SQLite-first architecture to PostgreSQL plus managed object storage for production scale.

Introduce background workers for document parsing and AI generation so long-running jobs do not block web requests.

Replace regex-only prompt-injection detection with layered content isolation, document sanitization, model-side instruction boundaries and adversarial evaluation.

Add stronger semantic contradiction detection, policy-family identifiers and configurable precedence rules with explicit supersession relationships.

Add richer observability: structured logs, metrics, traces, API latency, model cost and failure dashboards.

Add stronger tenant isolation, SSO/OIDC, MFA and managed secret storage for enterprise deployment.

Extend analytics with role-level competency heatmaps, manager alerts and learning-outcome trends.

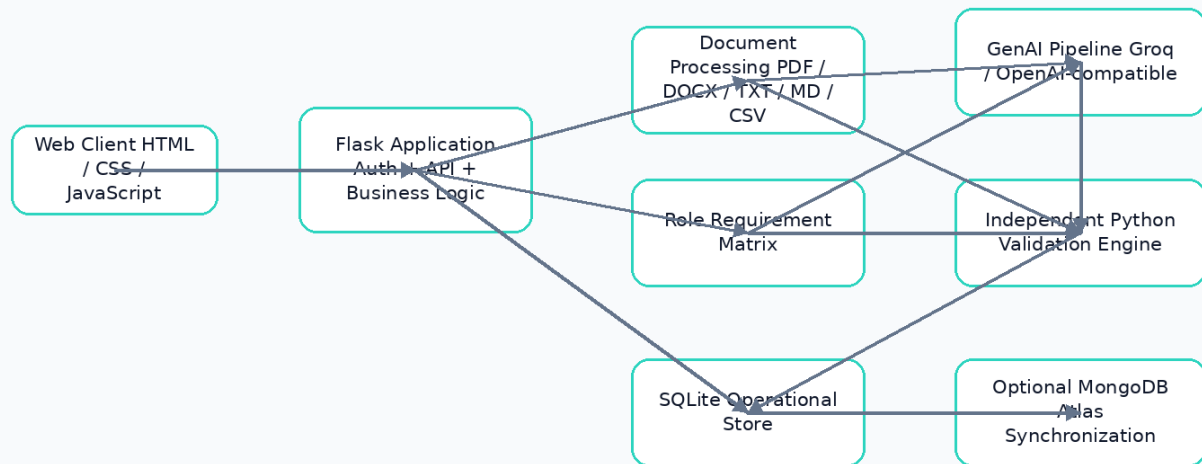
28. Conclusion

SkillSprint AI demonstrates a practical pattern for using Generative AI in a domain where correctness and traceability matter. The system does not rely on an LLM alone; it combines approved evidence, a structured requirement matrix, schema-constrained generation, deterministic Python validation and human review.

The supplied implementation provides the core architecture required by the SRS and contains substantial supporting evidence in its source code, dataset, security report, tests and operational documentation. The remaining work is primarily evaluation evidence and final polish: complete the ten-role evidence set, retain 100+ comparison rows, execute tests in the correct environment, deploy the application, record the required demonstration video and standardize documentation naming.

Appendix A — Architecture Diagrams

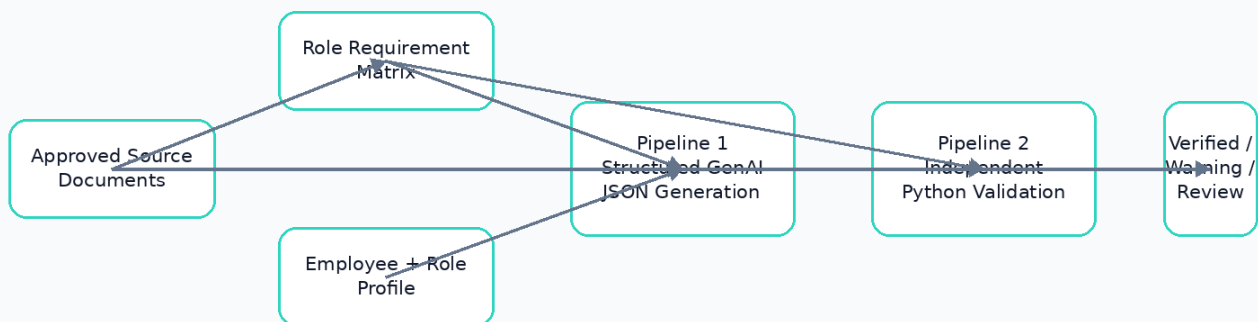
SkillSprint AI — High-Level Architecture



Design principle: AI generates; deterministic Python verifies.

Figure A1. High-level system architecture.

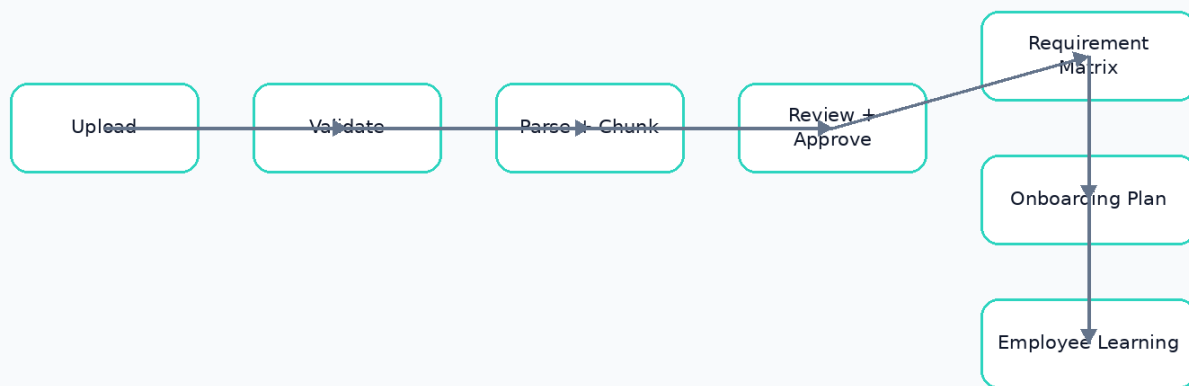
Dual-Pipeline Verification Model



Validation compares structured attributes and source references—not exact natural-language wording.

Figure A2. Dual-pipeline generation and verification.

Document-to-Learning Data Flow



Every mandatory learning item is expected to remain linked to an approved source.

Figure A3. Document-to-learning data flow.

Appendix B — Supplied Dataset and Roles

Dataset category	Observed coverage
Role descriptions	10 DOCX role-description files
Policies	Company handbook, HR, information security versions, data privacy
SOPs	Customer escalation, sales discounts, IT provisioning, bug triaging, finance reconciliation
FAQs	IT/security FAQ and HR/leave FAQ
Adversarial content	ADV-01_Hidden_Memo.docx

Supported roles

- Sales Executive
- Customer Support Executive
- HR Executive
- Finance Associate
- Operations Coordinator
- Marketing Executive
- Software Support Engineer
- Branch Manager
- Data Analyst
- Team Leader

Appendix C — Representative API Surface

Area	Endpoint	Purpose
Authentication	POST /api/auth/login	Create authenticated session
Registration	POST /api/auth/register	Self-register a user
Users	POST /api/admin/employees	Create employee/user
Documents	POST /api/documents/upload	Upload and process source document
Documents	POST /api/documents/approve	Approve source document
Generation	POST /api/generate/	Generate and validate onboarding plan
Plans	GET /api/plans/	View plan and validation report
Review	POST /api/plans/review	Approve/reject/override plan
Employee	GET /api/employee/dashboard	Load employee learning dashboard
Learning	POST /api/learning-attempts	Record quiz/learning result
Recommendations	GET /api/employees/recommendations	Return adaptive recommendations
Impact	GET /api/impact-analysis	List policy impacts
Regeneration	POST /api/impact-analysis/regenerate	Regenerate affected plan
Reports	GET /api/export/compliance.csv	Export compliance CSV
Reports	GET /api/export/compliance.xlsx	Export Excel-compatible report
Reports	GET /api/export/compliance.pdf	Export PDF report

Appendix D — Core Data Entities

Entity	Purpose
documents	Approved/pending organizational source documents and metadata
chunks	Traceable document sections/chunks
requirements	Role Requirement Matrix records
requirement_metadata	Classification, competency and applicability metadata
employees	Employee role and onboarding profile
employee_profiles	Location, competencies and training status
plans	Generated onboarding JSON and validation report
generation_logs	Provider, model, prompt version, attempts and errors
plan_reviews	Human review decisions and overrides
completions	Requirement-level employee completion state
learning_attempts	Quiz/learning scores and status
policy_impacts	Links changed documents to affected plans/requirements
users	Authentication accounts and role assignments
document_metadata	Expiry, family and precedence information
leave_requests	Employee leave workflow

Appendix E — Final Submission Readiness Checklist

Deliverable	Readiness
Project report/documentation	Present
Source code + requirements.txt	Present
Company document dataset	Present — 23 DOCX files in supplied dataset
Role Requirement Matrix	Present — 150 records in inspected DB snapshot
GenAI prompt templates	Present
Python validation pipeline	Present
GenAI/Python comparison evidence	Generate/retain final 100+ rows
Ten role onboarding evidence	Generate/retain final ten-role evidence set
Security testing report	Present — 16 documented security cases
Installation/execution guides	Present
Deployment URL	Pending in repository checklist
Mandatory demo video	Pending in repository checklist
Technical blog	Present
AI_USAGE.md	Present
Team contribution record	Confirm before final submission

— End of Professional Project Documentation —